

Agent Runtime 技术全景

Agent Runtime 技术调研 — Gateway（API 与模型访问治理）

一、关键问题

Agent 基础设施的 Gateway 层不像微服务网关那样已有成熟共识。微服务网关（Kong、APISIX、Nginx）解决的是「谁调用哪个服务、怎么限流、怎么鉴权」这三个问题，而 Agent 时代的 Gateway 面临着**完全不同的问题集**：

1. **模型不是服务，API 不是统一的**。微服务有 REST/gRPC 统一协议，LLM 却是 OpenAI、Claude Messages、Gemini 三套互不兼容的 API 格式。Gateway 必须做协议转换，否则 Agent 每接一个新模型就要重写适配代码。
2. **Agent 的调用路径不是「客户端→服务端」，而是「Agent→Gateway→Model Provider」**。Gateway 夹在 Agent 和模型之间，要同时处理 Agent 侧的身份认证（谁在用 Agent）和 Provider 侧的 API Key 管理（谁在付模型账单），形成双层认证链。
3. **Agent-to-Agent (A2A) 通信需要不同的网关语义**。当多个 Agent 相互调用时，需要的不再是「模型路由」，而是「Agent 发现 + 能力声明 + 直连通信」。这和 LLM API 网关是完全不同的技术栈。
4. **MCP 工具调用带来了新的网关需求**。Agent 调用 MCP Server 执行工具，这些工具调用同样需要认证、限流、审计和版本管理——但 MCP 协议的网关治理方案才刚刚出现。
5. **成本是硬约束**。一个 Agent 会话可能产生数十次模型调用，每次调用的 Token 消耗、延迟和成本需要实时可见、按团队归属、自动控预算。传统 API 网关的「QPS 限流」在 Token 经济下失效——一次慢思考可能消耗 64K Token，等价于数百次普通调用。

这五个问题构成了 Agent Gateway 的研究主线。目前没有单一项目能完整覆盖——事实上的格局是 **LiteLLM 统治模型访问层**，**Higress 和 Envoy AI Gateway 争夺基础设施层**，**MCP Gateway Registry 试图定义 Agent 资产治理层**。

二、技术方向

方向一：LLM 统一路由

核心问题： Agent 代码只应写一次 `model="gpt-4o"`，换模型时不该改代码。Gateway 负责根据请求特征（成本、延迟、可用性）将流量路由到最合适的 Provider 和模型，提供负载均衡、故障转移和 A/B 测试。

代表项目：

- **LiteLLM** (Python, 167k 行) 是该方向的事实标准。其核心设计是「OpenAI 兼容 API 作为通用界面」——100+ Provider 的 API 差异被封装在 Provider 适配层内，对外统一暴露 `/chat/completions` 端点。LiteLLM Proxy 进一步提供负载均衡（轮询/权重/最少延迟）、RPM/TPM 限流、PostgreSQL 持久化的成本追踪，以及 Virtual Key 体系实现多租户预算隔离。它是 LangChain 等框架的默认模型适配层，生态绑定极深。
- **New API** (Go, 40.9k stars) 是国内最活跃的 LLM 网关，基于已停止维护的 One API (MIT) 演进而来。其核心差异在于**信道路由机制**：每个模型-Provider 组合是一个独立「信道」，支持加权随机选择、失败自动重试、按模型粒度配置并发上限。前端为自研 AGPLv3 授权，后端 Go 实现。
- **Higress** (Go/C++/Rust, 阿里开源) 在网关层面实现了 `ai-load-balancer` 插件，基于 FNV-1a 一致性哈希做多模型负载均衡，配合 `model-router` 插件实现语义路由——根据请求内容而非仅仅模型名进行路由决策。这是目前唯一在 Envoy 数据面实现 FNV-1a 一致性哈希的 AI 网关方案。
- **Envoy AI Gateway** (Go/C++, Envoy Foundation) 在 Envoy Filter 层面拦截 LLM 流量，通过与 Service Mesh 集成获得分布式追踪和指标能力。与 LiteLLM 「应用层代理」不同，它是「网络层透明拦截」——Agent 代码无需感知 Gateway 存在。

关键差异总结：

维度	LiteLLM	New API	Higress	Envoy AI Gateway
运行层次	应用层 (Python Proxy)	应用层 (Go Proxy)	基础设施层 (Envoy Wasm)	基础设施层 (Envoy Filter)
Provider 覆盖	100+	主流厂商 + Midjourney/Suno	100+ 内置预设	Provider 抽象层
路由策略	轮询/权重/最少延迟	加权随机 + 失败重试	一致性哈希 + 语义路由	xDS 动态配置

维度	LiteLLM	New API	Higress	Envoy AI Gateway
部署方式	pip/docker	docker-compose	Helm/K8s CRD	Envoy xDS

方向二：协议转换（OpenAI/Claude/Gemini 互转）

核心问题：不同 LLM Provider 的 API 格式互不兼容——OpenAI 用 `messages` 数组 + `tools`，Claude 用 `messages` 但 `system` 是顶层字段，Gemini 用 `contents` + `parts`。Agent 开发者如果直接对接每家 API，切换成本极高。

技术路线存在分歧：是要求所有 Provider 适配一种格式（OpenAI 兼容），还是做双向格式转换？

路线 A：OpenAI 作为统一格式（LiteLLM、Higress）

LiteLLM 的策略是让所有 Provider 输入输出都转换为 OpenAI ChatCompletion 格式。Agent 代码只需写 OpenAI SDK，Gateway 负责翻译。这是目前最广泛采用的路线，因为生态惯性——LangChain、AutoGPT、CrewAI 等框架都基于 OpenAI SDK 构建。

Higress 的 `ai-proxy` Wasm 插件采用同样思路，但在 Envoy 数据面实现协议转换——意味着转换逻辑在请求流经 Envoy 时完成，无需独立代理进程。配合流式处理能力，SSE 响应可以逐 chunk 转换而不需要缓冲完整响应体。

路线 B：双向格式互转（New API）

New API 的 `relay` 中继层是唯一实现 OpenAI-Claude-Gemini 三者之间**双向**格式转换的网关。具体实现：- OpenAI <-> Claude Messages：`system` 字段在顶层 vs `messages[0]` 之间转换，`tool_use` 块与 `tool_calls` 之间映射 - OpenAI <-> Gemini：`contents` / `parts` 结构与 `messages` 数组互转 - Reasoning Effort 配置：通过模型名称后缀（如 `o3-mini-high`、`claude-3-7-sonnet-thinking`）控制推理深度，无需修改请求参数

New API 的格式转换比 LiteLLM 更激进——LiteLLM 要求用户使用 OpenAI 格式，而 New API 允许用户用 Claude 原生格式调用、网关自动转为 Gemini 格式。这在多 Agent 协作场景（Agent A 用 Claude SDK，Agent B 用 Gemini SDK，通过同一网关互操作）中有独特价值。

路线 C：桌面端配置级切换（CC Switch）

CC Switch 走的是完全不同的路线——不是在网关层转换协议，而是在桌面端统一管理多个工具的 API 配置。用户配置一个 API Key，CC Switch 自动写入 Claude Code、Codex、Gemini CLI 等所有工具的配置文件。内置 50+ Provider 预设，一键切换。这不是「协议转换」，而是「配置同步」——但对于个人开发者来说解决问题的方式更直接。

趋势判断： 路线 A（OpenAI 兼容）已经成为事实标准，但路线 B（双向转换）在 Agent-to-Agent 协作场景中会越来越重要。当多 Agent 系统需要跨模型通信时，原生格式支持比统一格式转换更可靠。

方向三：认证与流控

核心问题： Gateway 夹在 Agent 和模型之间形成双层认证——Agent 侧需要验证「谁在调用」（用户/Agent 身份），Provider 侧需要管理「谁在付钱」（API Key 池）。流控维度从传统 QPS 扩展为「RPM + TPM + 成本预算」三维体系。

认证方案对比：

项目	Agent 侧认证	Provider 侧认证	多租户隔离
LiteLLM	Virtual Key 体系	Master Key + Provider API Keys	按 user/team 预算隔离
New API	OIDC/Discord/Telegram/LinuxDO 多渠道登录	Token 分组 + 模型粒度限制	预充值 + 额度分配
Higress	JWT/OIDC/OAuth2/OPA 策略链	API Key 池轮转	WAF + CC 防护
MCP Gateway Registry	6 大 IdP (Keycloak/Entra ID/Okta/Auth0/Cognito/PingFederate)	4 种凭证类型并发	路径命名空间隔离

流控策略对比：

- 传统 QPS 限流**（Envoy AI Gateway、Kgateway）：依赖 Envoy 原生限流能力，适合 API 层面的流量整形，但对 Token 消耗类场景不够精细。
- RPM/TPM 限流**（LiteLLM、New API）：按每分钟请求数和 Token 数双重限流。LiteLLM 还支持请求排队——超限时自动排队等待而非直接拒绝。这是目前 LLM 网关的主流流控模式。
- Token 粒度流控**（Higress）：区别于 QPS 和 RPM，直接按 Token 消耗量做配额管理。配合 ai-context-limit 插件强制限制上下文窗口大小，防止 Agent 单次调用消耗过多 Token。这是最精细的限流方案。
- 成本预算控**（LiteLLM、New API）：按 dollar 金额设定预算阈值，超出后告警或阻断。LiteLLM 支持按 user/team/tag 三维度汇总报表，New API 支持多币种分列显示和缓存命中独立计价。

MCP Gateway Registry 的认证方案值得特别关注。 它是唯一实现「多 IdP 联邦 + 多凭证类型并发」的网关——同一个 Agent 调用可以用 Session Cookie（浏览器场景）、IdP JWT（服务间调用）、静态 Token（CI/CD）或联邦 Token（跨组织）四种方式认证。这种灵活性在 Agent 生产化落地中至关重要。

方向四：MCP 网关注册

核心问题： MCP 协议定义了 Agent 如何调用工具，但没有定义「谁来管理这些工具」。当组织内有数十个 MCP Server（数据库查询、文件操作、API 调用、代码执行等），Agent 需要一种机制来发现可用的工具、认证访问权限、审计调用记录、管理版本升级。

目前有三种 MCP 治理路线：

路线 A：MCP 专用注册中心（MCP Gateway Registry）

MCP Gateway Registry 从 MCP 网关起步，已演变为通用 AI 资产注册中心。核心能力： - **MCP Server 注册与发现**：MCP Server 注册到注册中心，Agent 通过统一 Nginx 网关访问，无需单独配置端点 - **虚拟 MCP Server 聚合**：将多个后端 MCP Server 的工具合并为一个统一端点，支持版本路由（多版本并行 + header 驱动测试 + 即时回滚） - **Agent 注册与 A2A 通信**：注册 AI Agent，Agent 通过注册中心发现后建立点对点直连 - **Skills 管理**：注册 SKILL.md 技能文件，自动安全扫描，供 Agent 按需加载 - **自服务工作流**：draft -> approval -> active 生命周期，支持人工审批或 CI/CD 自动化

技术栈：Python/FastAPI + Nginx 反向代理 + MongoDB/DocumentDB，支持 SSE 和 Streamable HTTP 双传输协议。它实质上已经是一个「AI 资产 Kubernetes」——不管理容器，而是管理 MCP Server、Agent、Skills 这些 AI 原生资产。

路线 B：网关内嵌 MCP Server（Higress）

Higress 的路线是把 MCP Server 作为 Wasm 插件部署在网关中。每个 MCP 工具调用经过 Higress 即获得： - **统一认证**：继承网关的 JWT/OIDC 认证链 - **Token 级流控**：工具调用也纳入 Token 消耗统计 - **全链路审计**：每次工具调用记录在 Higress 审计日志中 - **动态热更新**：Wasm 插件独立升级，不影响其他 MCP Server

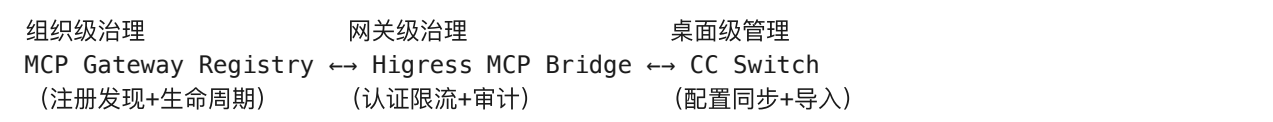
配套的 openapi-to-mcp 工具可将已有 OpenAPI 接口自动转为 MCP Server，降低存量 API 接入 MCP 生态的门槛。

路线 B 的优势是与 API 网关深度整合——MCP 工具和 REST API 共享同一套治理基础设施。但局限是 MCP Server 必须用 Wasm 编写（Go/Rust/JS 编译），当前生态中的 Python/TypeScript MCP Server 无法直接部署。

路线 C：桌面端 MCP 管理（CC Switch）

CC Switch 在桌面端统一管理 Claude Code、Codex、Gemini CLI 等工具的 MCP Server 配置。支持双向同步（修改配置文件自动回填数据库，切换工具自动写入配置）和 Deep Link 一键导入。这是「个人开发者」视角的 MCP 治理——不解决组织级问题，但让开发者在多工具间切换时不必手动编辑 JSON。

三种路线的互补关系：



三个项目目前互不集成，但逻辑上是互补的。一个完整的 MCP 治理方案需要注册中心（发现）+ 网关（流量治理）+ 客户端（配置体验）三层协同。

AgentGateway 值得单独讨论。它不走 LLM API 网关路线，也不是 MCP 网关——它是 **A2A 协议原生** 的 Agent 通信网关，解决 Agent 间互调用的发现、路由和认证。它比 MCP Gateway Registry 更聚焦 Agent-to-Agent 场景，但受限于 A2A 协议的采纳度。当多 Agent 协作从「编排框架内调用」演进为「跨组织 Agent 通信」时，这类 A2A 专用网关的价值会凸显。

三、趋势与争议

趋势一：AI 网关与 API 网关的融合不可逆

Higress 已经展示了「流量网关 + 微服务网关 + 安全网关 + AI 网关」四合一的可行性，Envoy AI Gateway 也在推进 Service Mesh + AI 的融合。未来 Agent 基础设施中的「网关」不会是一层独立组件，而是现有网关平面上的 AI 专用扩展。

趋势二：MCP 治理从「协议层」向「平台层」迁移

MCP 协议只定义了 Agent-Tool 的通信格式，没有定义治理语义。MCP Gateway Registry、Higress MCP Bridge、CC Switch 分别从注册中心、网关和桌面端填补这一空白。但三者的割裂意味着：目前没有一个统一的 MCP 治理标准。Anthropic 官方的 MCP Registry API 可能成为这一标准的起点。

趋势三：从「模型网关」到「AI 资产网关」

MCP Gateway Registry 的演进路径（MCP Server -> Agent -> Skills -> Custom Entities）最具代表性。Gateway 不再是简单的请求转发，而是 AI 资产的全生命周期管理——注册、审批、版

本、安全扫描、审计、退役。

争议一：应用层 vs 基础设施层——谁该做 AI 网关？

LiteLLM（Python 应用层）和 Envoy AI Gateway（C++ 基础设施层）代表了两种哲学： - **应用层派**：灵活，Provider 适配快，Python 生态丰富，适合快速迭代 - **基础设施层派**：性能高，与 Service Mesh 天然集成，零代码侵入，适合规模化部署

目前 LiteLLM 在生态绑定上占优（LangChain 依赖），Envoy AI Gateway 在性能上占优（C++ vs Python）。但长期看，应用层网关的 Provider 覆盖优势可能会被基础设施层追赶——Envoy AI Gateway 的 Provider 抽象层设计上就是可扩展的。

争议二：OpenAI 兼容 API 是否应该成为唯一标准？

LiteLLM 和 Higress 坚持「所有请求统一为 OpenAI 格式」，New API 走双向转换路线。后者的理由是：当多个 Agent 使用不同原生 SDK（Claude SDK、Gemini SDK）协作时，强制统一格式会丢失原生特性（如 Claude 的 extended thinking、Gemini 的 grounding）。但双向转换的维护成本更高——每增加一个新模型格式，转换矩阵呈 $O(n^2)$ 增长。

争议三：MCP Server 应该独立托管还是嵌入网关？

Higress 选择将 MCP Server 嵌入网关（Wasm），MCP Gateway Registry 选择独立托管（Docker Compose / ECS / EKS）。前者的优势是统一治理平面，后者的优势是 MCP Server 可以用任意语言编写、独立扩展。目前生态中绝大多数 MCP Server 是 Python/TypeScript 实现，短期内 Wasm 路线受限语言生态。

争议四：rtk 算不算 Gateway？

严格来说，rtk（CLI 输出压缩代理）不是 API 网关，但它解决的问题属于同一层——Agent 和外部系统（Shell）之间的代理层。**如果把 Gateway 定义为「Agent 与外部世界的所有通信代理」，那么 rtk 就是一种特殊目的的 Gateway。**这个视角下，Agent Gateway 不只是 LLM API 代理，还包括 Shell 代理、文件系统代理、数据库代理等。当前生态中这类「非 LLM API 代理」是明显的空白地带。

四、项目全景表

项目	Stars	语言	定位	核心能力	代表场景
LiteLLM	25k+	Python	LLM API 统一网关	100+ Provider 适配、OpenAI 兼容、Virtual Key、RPM/TPM 限流、成本追踪	Agent 多模型切换、企业 AI Gateway
New API	40.9k	Go	LLM 聚合分发平台	多格式双向互转 (OpenAI/Claude/Gemini)、信道路由、多租户计费	国内模型聚合、API Key 分发
Higress	5k+	Go/C++/Rust	AI 原生 K8s API 网关	Wasm 插件、MCP Server 托管、Token 粒度流控、一致性哈希路由	企业 K8s 集群 AI 流量治理
Envoy AI Gateway	1k+	Go/C++	Envoy 层 AI 代理	Envoy Filter 拦截、Provider 抽象、xDS 动态配置、Service Mesh 集成	Service Mesh + AI 融合
Kgateway	500+	Go	K8s Gateway API AI 扩展	K8s CRD 驱动、AI BackendTrafficPolicy、多模型路由	K8s 原生 GitOps 管理 AI 流量
AgentGateway	200+	Rust	A2A Agent 通信网关	Agent 注册发现、A2A 协议路由、负载均衡、插件架构	Multi-Agent 间互调用
MCP Gateway Registry	500+	Python	AI 资产治理中枢	MCP Server 注册、虚拟聚合、Agent 注册/A2A、Skills 管理、OAuth2 SSO	企业 MCP 工具治理
CC Switch	112k+	Rust/TS	桌面 AI 工具配置管理	多工具 API 统一配置、MCP Server 管理、Skills/Prompts 同步、内置代理	个人多 AI 工具切换
RTK	5k+	Rust	CLI 输出压缩代理	12 种压缩策略、透明代理、Hook 引擎、Token 节省追踪	Agent 编码场景成本优化
CLIProxyAPI	< 100	Go	本地 CLI LLM 代理	轻量 API 转发、多 Provider、单进程运行	简单 CLI 工具 AI 赋能

注： Stars 数据为截至 2026-07-02 的近似值，CC Switch 的 112k+ 尤其值得关注——它表明「桌面端 AI 工具配置管理」是一个被严重低估的大众市场需求。

按技术路线归类

LLM 模型访问层（应用层代理）
└─ LiteLLM ← 事实标准，Python 生态

- └─ New API ← 国内最活跃, Go 生态
- 基础设施层 (网络层/Envoy 层)
 - └─ Higress ← 四合一 AI 原生网关, 阿里生产验证
 - └─ Envoy AI Gateway ← Envoy Foundation 官方, Service Mesh 原生
 - └─ Kgateway ← CNCF, K8s Gateway API 扩展
- Agent 资产治理层
 - └─ MCP Gateway Registry ← MCP Server + Agent + Skills 注册中心
 - └─ AgentGateway ← A2A 协议原生 Agent 通信网关
 - └─ CC Switch ← 桌面端 AI 工具统一配置
- 特殊目的代理
 - └─ RTK ← CLI 输出压缩, Token 节省 60-90%
 - └─ CLIProxyAPI ← 轻量本地 CLI LLM 代理

按成熟度归类

阶段	项目
生产就绪	LiteLLM, Higress (阿里生产数十万 QPS) , New API
快速成长	MCP Gateway Registry, CC Switch, RTK
早期阶段	Envoy AI Gateway, Kgateway, AgentGateway, CLIProxyAPI

五、开放问题与进一步研究方向

- A2A 与 MCP 在网关层的统一：** AgentGateway 专注 A2A, MCP Gateway Registry 专注 MCP。当 Agent 同时使用 A2A (Agent 通信) 和 MCP (工具调用) 时, 是否需要两套网关? 还是会有统一方案?
- 流式协议转换的可靠性和性能：** SSE 响应体的逐 chunk 解析在 Envoy C++ Filter 层和 Go 应用层的实现难度差异很大。不同网关对 data: [DONE] 信号、连接中断、chunk 乱序等异常情况的处理是否一致?
- Token 粒度流控的标准化：** 目前各网关的 Token 计数方式不统一 (有的按 API 返回的 usage 字段, 有的自己估算), 导致成本追踪口径不一致。行业是否需要 Token 计数的标准规范?
- MCP 网关的安全模型：** MCP 工具调用本质上是「让 LLM 决定执行什么代码/查什么数据」, 其安全风险远高于普通 API 调用。当前 MCP 网关 (MCP Gateway Registry、Higress MCP Bridge) 的安全能力是否足够? Prompt 注入 + MCP 工具调用的组合攻击如何防御?

5. **Gateway 的 Gateway:** rtk 代表了一类「非 LLM API 代理」的需求——Agent 需要代理 Shell 命令、文件读写、数据库查询、浏览器操作等。这些代理是否应该和 LLM API 网关统一？还是各管各的？